

Chương 4 đã trình bày quá trình phân tích, thiết kế, xây dựng và kiểm thử hệ thống. Trên cơ sở đó, chương này tập trung vào các giải pháp kỹ thuật có ý nghĩa nổi bật trong đề án. Các nội dung được lựa chọn không lặp lại phần mô tả chức năng, mà đi vào những bài toán có tính triển khai cao hơn: xây dựng trợ lý AI dựa trên học liệu nội bộ, kiểm soát quá trình làm bài kiểm tra, xử lý video bài giảng dung lượng lớn và tự động hóa ghi danh sau thanh toán. Mỗi giải pháp được trình bày theo ba phần: bài toán và vấn đề, giải pháp đã triển khai, kết quả đạt được và cách đánh giá.

0.1 Trợ lý AI hỏi đáp theo học liệu nội bộ

Trợ lý AI là điểm khác biệt quan trọng của hệ thống so với một LMS thông thường. Tuy nhiên, trong phạm vi đề án, mục tiêu không phải là tự huấn luyện mô hình ngôn ngữ từ đầu, mà là xây dựng một luồng hỏi đáp có khả năng bám vào học liệu do giáo viên cung cấp. Phần này trình bày toàn bộ quy trình vận hành của tính năng, từ lúc giáo viên tải học liệu lên hệ thống cho đến khi học sinh nhận được câu trả lời có nguồn dẫn.

0.1.1 Bài toán và vấn đề

Trong lớp học trực tuyến, học sinh thường xem lại bài giảng vào thời điểm giáo viên không thể trả lời ngay. Một trợ lý AI đặt trực tiếp trong màn hình học bài có thể giúp học sinh giải đáp nhanh các thắc mắc cơ bản. Tuy nhiên, nếu hệ thống chỉ gửi câu hỏi của học sinh tới một mô hình ngôn ngữ lớn, mô hình có thể trả lời dựa trên tri thức tổng quát, trong khi câu hỏi của học sinh lại phụ thuộc vào nội dung bài giảng cụ thể. Ví dụ, câu hỏi “phần này cần nhớ gì” chỉ có ý nghĩa khi hệ thống biết học sinh đang học bài nào và bài đó có tài liệu gì.

Bài toán đặt ra gồm ba thách thức kỹ thuật. Thách thức thứ nhất là học liệu trong hệ thống có nhiều dạng khác nhau, gồm mô tả bài học, nội dung nhập trực tiếp, tài liệu PDF, DOCX hoặc PPTX. Các dữ liệu này không thể đưa nguyên vẹn vào prompt vì có thể dài, không đồng nhất định dạng và vượt giới hạn ngữ cảnh của mô hình. Thách thức thứ hai là truy xuất phải ưu tiên bài học hiện tại, nhưng vẫn có khả năng dùng kiến thức liên quan trong cùng khóa học khi bài hiện tại thiếu ngữ cảnh. Thách thức thứ ba là hệ thống phải xử lý trường hợp không có tài liệu phù hợp. Với một hệ thống giáo dục, câu trả lời “có vẻ đúng” nhưng không dựa trên học liệu là rủi ro nghiêm trọng hơn so với việc trả lời rằng chưa đủ thông tin.

Do đó, đóng góp của phần này không nằm ở việc gọi API mô hình ngôn ngữ, mà nằm ở việc thiết kế một quy trình RAG đủ kiểm soát cho bối cảnh LMS. Quy trình này cần biến học liệu thành dữ liệu truy xuất được, chọn đúng đoạn liên quan, đưa ngữ cảnh vào prompt, bắt buộc câu trả lời có nguồn dẫn và ghi lại nguồn đã sử

dụng để kiểm tra sau này.

0.1.2 Giải pháp

Luồng vận hành của tính năng gồm hai giai đoạn lớn. Giai đoạn thứ nhất là chuẩn bị tri thức, diễn ra khi giáo viên thêm hoặc cập nhật học liệu. Giai đoạn thứ hai là hỏi đáp, diễn ra khi học sinh đặt câu hỏi trong màn hình học bài.

a, Chuẩn bị tri thức từ học liệu. Khi giáo viên thêm học liệu, backend lưu bản ghi `lesson_materials` với trạng thái xử lý. Nếu học liệu là nội dung văn bản nhập trực tiếp, hệ thống có thể đưa nội dung này vào bước đánh chỉ mục. Nếu học liệu là tệp PDF, DOCX hoặc PPTX, dịch vụ AI tải tệp về thư mục tạm và trích xuất văn bản.

Trong bước trích xuất, hệ thống ưu tiên sử dụng `MarkItDown`. Đây là thư viện chuyển đổi tài liệu sang văn bản/Markdown, giúp giữ lại nội dung chính của nhiều định dạng tài liệu phổ biến theo một đầu ra thống nhất. Việc dùng `MarkItDown` giúp giảm số lượng bộ xử lý riêng lẻ cần tự viết, đồng thời tạo văn bản đầu vào sạch hơn cho bước chunking. Tuy nhiên, không phải tệp nào cũng được `MarkItDown` xử lý tốt. Vì vậy, hệ thống có cơ chế dự phòng: dùng `pypdf` để đọc PDF, `python-docx` để đọc DOCX và `python-pptx` để đọc PPTX. Với PDF, nội dung được gán chỉ số trang; với PPTX, nội dung được gán chỉ số slide. Nhờ đó, khi một đoạn được truy xuất, hệ thống vẫn có thể lần ngược về nguồn ban đầu của đoạn kiến thức.

Sau khi trích xuất thành công, hệ thống cập nhật `extracted_text`, chuyển trạng thái học liệu sang `ready` và xóa lỗi xử lý cũ nếu có. Nếu không trích xuất được nội dung hoặc tệp không hợp lệ, trạng thái chuyển sang `failed` và hệ thống lưu thông báo lỗi rút gọn. Cách tổ chức này giúp giáo viên biết học liệu nào đã sẵn sàng cho AI Tutor, học liệu nào cần tải lại hoặc chỉnh sửa.

b, Chiến lược chunking. Văn bản học liệu thường dài và có cấu trúc không đều. Nếu chia theo số ký tự cố định, một khái niệm có thể bị cắt giữa câu. Nếu giữ nguyên toàn bộ tài liệu, truy xuất sẽ kém chính xác và prompt trở nên quá dài. Hệ thống sử dụng `RecursiveCharacterTextSplitter` để chia văn bản theo các mức ưu tiên: đoạn văn, dòng, câu, từ và cuối cùng là ký tự. Kích thước mặc định của mỗi chunk là 1000 ký tự, với phần chồng lấn 150 ký tự giữa hai chunk liên tiếp.

Phần chồng lấn có vai trò giữ lại ngữ cảnh ở ranh giới giữa hai đoạn. Ví dụ, nếu một định nghĩa nằm cuối chunk trước và ví dụ minh họa nằm đầu chunk sau, overlap giúp hai đoạn vẫn giữ được một phần liên hệ. Mỗi chunk được tính mã băm SHA-256 từ nội dung. Khi một học liệu được xử lý lại, hệ thống xóa các chunk cũ

theo `material_id` rồi ghi lại các chunk mới, tránh tình trạng cơ sở tri thức còn chứa phiên bản cũ của tài liệu.

c, Embedding và lưu trữ vector. Mỗi chunk được chuyển thành vector embedding bằng provider OpenAI-compatible. Trong cấu hình hiện tại của đề án, mô hình embedding là `text-embedding-3-small`, số chiều vector là 1536. Trước khi lưu, hệ thống kiểm tra số chiều vector trả về có khớp với cấu hình hay không. Nếu số chiều không đúng, quá trình xử lý bị dừng và học liệu được đánh dấu lỗi. Đây là kiểm tra cần thiết vì cơ sở dữ liệu vector yêu cầu số chiều nhất quán; nếu bỏ qua, truy vấn pgvector có thể lỗi hoặc trả kết quả không đáng tin cậy.

Các chunk được lưu vào bảng `knowledge_chunks`. Bảng này không chỉ lưu nội dung và vector, mà còn lưu khóa học, bài học, tài liệu gốc, loại nguồn, thứ tự chunk, tên provider, tên mô hình embedding và thời điểm đánh chỉ mục. Nhờ đó, mỗi kết quả truy xuất đều có thể truy ngược về bài học hoặc tài liệu cụ thể.

d, Hỏi đáp theo luồng RAG. Khi học sinh đặt câu hỏi trong bài học, frontend gửi câu hỏi kèm `course_id` và `lesson_id`. Dịch vụ AI tạo embedding cho câu hỏi bằng cùng mô hình embedding đã dùng khi đánh chỉ mục học liệu. Sau đó, hệ thống truy vấn các chunk thuộc cùng khóa học bằng pgvector. Độ tương đồng cơ sở được tính từ khoảng cách vector; điểm càng cao thì chunk càng gần câu hỏi về mặt ngữ nghĩa.

Ở mức triển khai, backend nhận câu hỏi của học sinh qua API gửi tin nhắn trong phiên chat, lưu tin nhắn người dùng vào bảng `tutor_chat_messages`, sau đó gọi dịch vụ AI nội bộ. Payload gửi sang dịch vụ AI có cấu trúc rút gọn như sau:

```
{
  "course_id": "...",
  "lesson_id": "...",
  "question": "...",
  "chat_history": [
    { "role": "user", "content": "..." },
    { "role": "assistant", "content": "..." }
  ]
}
```

Trong đó, `course_id` giới hạn phạm vi truy xuất trong một khóa học, `lesson_id` giúp ưu tiên bài học hiện tại, `question` là câu hỏi mới của học sinh, còn `chat_history` chứa tối đa các lượt trao đổi gần nhất để giữ mạch hội thoại. Dịch vụ AI trả về câu trả lời cùng danh sách nguồn dẫn:

```
{
  "answer": "... [1]",
  "citations": [
    {
      "chunk_id": "...",
      "rank": 1,
      "score": 0.82,
      "source_title": "..."
    }
  ]
}
```

Cấu trúc này làm rõ ranh giới giữa backend LMS và dịch vụ AI. Backend chịu trách nhiệm xác thực học sinh, quản lý phiên chat và lưu citation; dịch vụ AI chịu trách nhiệm truy xuất chunk, dựng prompt và sinh câu trả lời.

Điểm khác biệt của hệ thống là truy xuất có ưu tiên bài học hiện tại. Với mỗi chunk c , câu hỏi q và bài học hiện tại l , điểm xếp hạng được tính như sau:

$$\text{score}(c, q, l) = \text{sim}(c, q) + \text{boost}(c, l) \quad (1)$$

Trong đó $\text{sim}(c, q)$ là độ tương đồng cơ sở giữa chunk và câu hỏi. Thành phần $\text{boost}(c, l)$ bằng 0.05 nếu chunk thuộc bài học hiện tại và bằng 0 trong trường hợp còn lại. Sau khi sắp xếp theo điểm đã cộng boost, hệ thống vẫn kiểm tra ngưỡng tương đồng cơ sở 0.55. Điều này giúp các đoạn trong bài học hiện tại được ưu tiên, nhưng một đoạn kém liên quan không được chọn chỉ vì nó thuộc cùng bài. Hệ thống lấy tối đa 6 đoạn ngữ cảnh cho mỗi câu hỏi.

e, Prompt và sinh câu trả lời. Sau khi truy xuất, hệ thống xây dựng prompt theo hai nhánh. Nếu tìm được ngữ cảnh phù hợp, prompt chứa tên khóa học, tên bài học, mô tả bài học, số lượng tài liệu của bài và các đoạn ngữ cảnh đã truy xuất. Mô hình chat được cấu hình theo provider OpenAI-compatible, trong môi trường hiện tại sử dụng `deepseek-v4-flash`. Tham số `temperature` được đặt ở mức 0.2, tức là mô hình được định hướng tạo câu trả lời ít ngẫu nhiên hơn, phù hợp với bài toán hỏi đáp theo học liệu nội bộ. Prompt yêu cầu mô hình chỉ dùng ngữ cảnh được cung cấp để trả lời câu hỏi chuyên môn và bắt buộc trích dẫn nguồn bằng ký hiệu như [1], [2].

Nếu không tìm thấy đoạn tài liệu đủ liên quan, hệ thống không gửi một prompt mở để mô hình tự suy diễn. Prompt trong trường hợp này yêu cầu mô hình nói rõ

rằng tài liệu hiện tại chưa cung cấp đủ thông tin và gợi ý học sinh hỏi lại cụ thể hơn hoặc kiểm tra tài liệu bài học. Nếu bài học hiện tại chưa có tài liệu, prompt cũng nhắc mô hình không được dùng tài liệu của bài khác để giả vờ trả lời cho bài hiện tại.

f, Lưu phiên chat và nguồn dẫn. Kết quả trả về gồm câu trả lời và danh sách citations. Mỗi citation lưu định danh chunk, định danh tài liệu, thứ hạng, điểm số, đoạn trích ngắn, tiêu đề nguồn, bài học liên quan và loại nguồn. Backend kiểm tra các chunk thuộc đúng khóa học trước khi lưu citation vào CSDL. Với chế độ streaming, hệ thống gửi sự kiện citations trước, sau đó gửi dần nội dung câu trả lời qua Server-Sent Events. Cách làm này giúp giao diện có thể hiển thị nguồn tham chiếu sớm, đồng thời người học không phải chờ toàn bộ câu trả lời được sinh xong mới thấy phản hồi.

0.1.3 Kết quả đạt được và đánh giá

Giải pháp đã hình thành một luồng hỏi đáp có kiểm soát nguồn dữ liệu. Hệ thống không chỉ trả lời bằng văn bản, mà còn lưu lại các nguồn đã được dùng để tạo câu trả lời. Với học sinh, citation giúp các em quay lại đúng tài liệu liên quan. Với giáo viên hoặc người quản trị, citation giúp kiểm tra xem câu trả lời có dựa trên học liệu phù hợp hay không.

Để đánh giá chất lượng truy xuất, đề án sử dụng hai chỉ số chính là *Recall@K* và *Mean Reciprocal Rank*. Với một tập câu hỏi kiểm thử, giả sử G là tập chunk đúng được gán nhãn thủ công và C_K là tập K chunk đầu tiên hệ thống truy xuất được, *Recall@K* được tính như sau:

$$\text{Recall@K} = \frac{|C_K \cap G|}{|G|} \quad (2)$$

Chỉ số này đo khả năng hệ thống lấy được các đoạn học liệu cần thiết. Bên cạnh đó, *Mean Reciprocal Rank* đánh giá vị trí xuất hiện của chunk đúng đầu tiên:

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i} \quad (3)$$

Trong đó N là số câu hỏi kiểm thử và rank_i là vị trí của chunk đúng đầu tiên đối với câu hỏi thứ i . Nếu chunk đúng thường đứng ở vị trí đầu, MRR sẽ cao. Hai chỉ số này phù hợp với bài toán của đề án vì chất lượng câu trả lời phụ thuộc trực tiếp vào việc truy xuất đúng học liệu trước khi sinh câu trả lời.

Ngoài truy xuất, hệ thống còn đánh giá tỷ lệ câu trả lời có citation bằng cách

tách câu và kiểm tra ký hiệu trích dẫn. Nếu câu trả lời có nhiều mệnh đề kiến thức nhưng không gắn nguồn, kết quả này cho thấy prompt chưa đủ chặt hoặc mô hình chưa tuân thủ tốt yêu cầu trích dẫn. Công cụ đánh giá RAG của dịch vụ AI đã hiện thực các phép đo này, gồm recall truy xuất, MRR, tỷ lệ câu có citation, độ trễ trung bình, số đoạn ngữ cảnh trung bình và số lần sinh câu trả lời thất bại. Nhờ đó, việc đánh giá AI Tutor không chỉ dựa trên cảm nhận khi đọc câu trả lời, mà có các chỉ số kiểm tra được.

0.2 Kiểm soát quá trình làm bài kiểm tra

Hệ thống bài kiểm tra là một phần quan trọng của LMS vì nó liên quan trực tiếp đến đánh giá kết quả học tập. Trong đồ án, bài kiểm tra không chỉ là một biểu mẫu câu hỏi đơn giản, mà được thiết kế để phục vụ nhiều bối cảnh học tập khác nhau: làm thử công khai, kiểm tra định kỳ trong khóa học và quiz tự luyện sau bài giảng. Phần này trình bày cách hệ thống tổ chức vị trí sử dụng bài kiểm tra, kiểm soát lượt làm và bảo toàn bài làm của học sinh trong các tình huống thực tế như thoát trang, mất mạng hoặc quay lại làm tiếp.

0.2.1 Bài toán và vấn đề

Trong thực tế vận hành một lớp học trực tuyến, giáo viên thường dùng bài kiểm tra ở ba vị trí khác nhau. Thứ nhất là bài luyện tập công khai, cho phép học sinh chưa mua khóa học làm thử một số đề để tự đánh giá năng lực. Thứ hai là bài kiểm tra gắn với khóa học, được dùng như bài thi định kỳ hoặc bài tập giao thêm cho cả lớp. Thứ ba là quiz gắn với một bài học cụ thể, giúp học sinh tự luyện ngay sau khi học xong một phần kiến thức. Ba trường hợp này có cùng nội dung lõi là đề kiểm tra, câu hỏi, đáp án và bài làm, nhưng khác nhau về quyền truy cập, thời gian mở, số lượt làm và vị trí hiển thị trong trải nghiệm học tập.

Nếu tách ba trường hợp trên thành ba mô hình dữ liệu riêng, hệ thống sẽ bị lặp logic tạo đề, lưu đáp án, chấm điểm và thống kê kết quả. Ngược lại, nếu gom tất cả vào một loại bài kiểm tra duy nhất mà không biểu diễn rõ vị trí sử dụng, hệ thống khó kiểm soát quyền truy cập và khó mở rộng. Ngoài ra, bài làm của học sinh phải được bảo toàn trong các tình huống không lý tưởng. Khi đang làm bài, học sinh có thể mất kết nối mạng, đóng nhầm trình duyệt hoặc quay lại sau một khoảng thời gian. Hệ thống cần cho phép tiếp tục lượt làm đang dở nếu lượt làm đó vẫn hợp lệ, đồng thời không được tạo thêm lượt làm mới ngoài số lần cho phép.

0.2.2 Giải pháp

Giải pháp được thiết kế quanh ba khái niệm chính: `assessment`, `assessment_placement` và `submission`. `Assessment` lưu nội dung gốc của bài kiểm tra, gồm tiêu đề, môn học, khối lớp, loại bài, hình thức chấm điểm, thời gian

làm bài, các phần và các câu hỏi. `Assessment_placement` mô tả vị trí sử dụng của bài kiểm tra. `Submission` lưu một lượt làm bài cụ thể của học sinh. Cách tách này cho phép một đề kiểm tra được tái sử dụng ở nhiều bối cảnh, trong khi quyền truy cập và quy tắc mở bài được kiểm soát tại `placement`.

a, Tách dữ liệu xem trước và dữ liệu phòng làm bài. API xem trước chỉ trả về metadata của bài kiểm tra như tiêu đề, thời gian làm bài, số lượt tối đa, trạng thái mở và thông tin vị trí sử dụng. Nội dung câu hỏi không được trả về ở bước này. Chỉ khi học sinh bắt đầu hoặc tiếp tục một lượt làm hợp lệ, hệ thống mới trả về workspace gồm section, item, đáp án đã lưu và thời gian còn lại. Cách tách này giúp giảm nguy cơ lộ đề trước khi học sinh thật sự vào bài.

b, Kiểm soát ba vị trí sử dụng. Placement công khai được dùng cho bài luyện tập mở, không gắn với khóa học hoặc bài học và cần có slug để học sinh truy cập từ khu vực công khai. Placement theo khóa học được dùng cho bài kiểm tra định kỳ hoặc bài tập giao thêm, bắt buộc gắn với một khóa học cụ thể và yêu cầu học sinh đã ghi danh. Placement theo bài học được dùng cho quiz tự luyện sau bài giảng, bắt buộc gắn với một bài học có kiểu `quiz`; khóa học được suy ra từ bài học để tránh gắn nhầm quiz sang khóa khác.

Khi giáo viên hoặc quản trị viên xuất bản bài kiểm tra, hệ thống kiểm tra tính hợp lệ của placement. Với placement theo khóa học và theo bài học, môn học và khối lớp của bài kiểm tra phải khớp với khóa học. Thiết kế này giúp giáo viên tạo một đề kiểm tra độc lập trước, sau đó quyết định sử dụng đề đó ở đâu mà không phải sao chép lại toàn bộ câu hỏi.

Hệ thống cũng phân biệt quyền của quản trị viên và giáo viên. Quản trị viên có thể thao tác ở phạm vi toàn hệ thống, còn giáo viên chỉ được gắn bài kiểm tra vào các khóa học hoặc bài học do mình quản lý. Điều này giữ được sự đơn giản của mô hình vai trò hiện tại, nhưng vẫn tránh việc giáo viên can thiệp vào nội dung của người khác.

c, Tạo và tiếp tục lượt làm. Khi học sinh bắt đầu làm bài, hệ thống kiểm tra ba nhóm điều kiện: người dùng đã đăng nhập, placement đang trong thời gian mở và học sinh có quyền truy cập. Với placement theo khóa học hoặc bài học, quyền truy cập được xác định thông qua bản ghi ghi danh. Sau đó, hệ thống kiểm tra xem học sinh đã có submission ở trạng thái `doing` hay chưa. Nếu có, hệ thống trả lại submission đang làm thay vì tạo mới. Nếu chưa có, hệ thống đếm số lượt đã làm và chỉ tạo submission mới khi chưa vượt quá `max_attempts`.

Ở phía giao diện, khi học sinh mở lại một bài kiểm tra đang làm dở, hệ thống nhận biết submission còn trạng thái `doing` và chuyển học sinh vào đúng phòng

làm bài cũ. Workspace được mở bằng cả `placementId` và `submissionId`, nên học sinh không thể tùy ý mở bài làm của người khác. Các đáp án đã lưu trước đó được dựng lại từ dữ liệu `submission`, giúp học sinh có thể tiếp tục làm nếu đã kịp lưu bài trước khi thoát trang hoặc mất kết nối.

Thời gian còn lại được tính theo hai ràng buộc: thời lượng làm bài và thời điểm đóng `placement`. Nếu bài kiểm tra có cả hai ràng buộc, hệ thống lấy mốc sớm hơn làm hạn cuối:

$$\text{deadline} = \min(\text{startTime} + \text{timeLimit}, \text{closeTime}) \quad (4)$$

$$\text{remainingSeconds} = \max(0, \text{deadline} - \text{now}) \quad (5)$$

Cách tính này xử lý được tình huống học sinh bắt đầu làm bài gần thời điểm đóng. Ví dụ, bài kiểm tra cho phép làm 60 phút nhưng đóng sau 20 phút nữa, hệ thống chỉ cho phép làm trong 20 phút còn lại.

d, Lưu đáp án và xử lý rủi ro khi làm bài. Mỗi lần học sinh bấm lưu nháp hoặc nộp bài, hệ thống kiểm tra `submission` thuộc đúng học sinh, trạng thái còn là `doing`, `placement` vẫn hợp lệ và học sinh vẫn có quyền truy cập. Sau đó, hệ thống kiểm tra từng câu trả lời. Với câu trắc nghiệm, các option được chọn phải thuộc đúng câu hỏi và không được lặp. Nếu câu trắc nghiệm chỉ cho phép một đáp án, hệ thống không cho phép gửi nhiều option. Với câu đúng sai, mỗi mệnh đề chỉ được trả lời một lần. Với câu tự luận, nội dung được lưu để giáo viên chấm sau.

Với trường hợp mất mạng hoặc đóng nhầm trình duyệt, hệ thống xử lý theo nguyên tắc rõ ràng: dữ liệu đã lưu vào `submission` thì được khôi phục khi học sinh quay lại, còn dữ liệu vừa nhập nhưng chưa gửi lên server thì không được coi là đã lưu. Vì vậy, giao diện có cảnh báo khi học sinh rời trang trong lúc còn thay đổi chưa lưu. Cách làm này không che giấu rủi ro mạng, nhưng giúp trạng thái bài làm nhất quán và dễ giải thích: muốn bảo toàn đáp án, học sinh cần lưu nháp hoặc nộp bài thành công.

Việc kiểm tra ở backend là cần thiết vì frontend không phải biên giới bảo mật. Một request có thể bị sửa bằng công cụ bên ngoài trình duyệt. Nếu backend không kiểm tra item và option, học sinh có thể gửi đáp án cho câu hỏi không thuộc bài kiểm tra hiện tại hoặc tạo dữ liệu bài làm không nhất quán.

Các API chính trong luồng làm bài được tách theo đúng trạng thái nghiệp vụ:

GET `/learning/assessment-placements/`

```

        :placementId
POST /learning/assessment-placements/
        :placementId/attempts
GET /learning/assessment-placements/
        :placementId/workspace
        ?submissionId=...
PUT /learning/assessment-submissions/
        :submissionId/answers
POST /learning/assessment-submissions/
        :submissionId/submit

```

API xem trước chỉ trả về thông tin tổng quan, API tạo lượt làm trả về submission đang làm hoặc submission mới, API workspace trả về dữ liệu phòng làm bài, còn hai API cuối phục vụ lưu nháp và nộp bài. Payload lưu đáp án được thiết kế dưới dạng union theo loại câu hỏi:

```

{
  "answers": [
    {
      "itemId": "...",
      "type": "mcq",
      "selectedOptionIds": ["..."]
    },
    {
      "itemId": "...",
      "type": "true_false",
      "selections": [
        { "optionId": "...", "selectedValue": true }
      ]
    },
    {
      "itemId": "...",
      "type": "numeric",
      "answerValue": 12.5
    },
    {
      "itemId": "...",
      "type": "essay",
      "answer": "..."
    }
  ]
}

```

```

    }
  ]
}

```

Nhờ payload có trường `type`, backend biết chính xác cần kiểm tra dữ liệu theo luật nào. Ví dụ, câu trắc nghiệm dùng `selectedOptionIds`, câu đúng sai dùng danh sách `selections`, câu số học dùng `answerValue`, còn câu tự luận dùng `answer`. Đây là phần triển khai quan trọng vì nó tránh việc gom mọi loại câu trả lời vào một trường văn bản chung rồi xử lý thủ công về sau.

e, Hết giờ, nộp bài và chấm điểm. Workspace luôn nhận thời gian còn lại từ backend. Khi hết giờ, giao diện tự gửi bài làm hiện tại lên hệ thống và gọi thao tác nộp bài. Backend vẫn kiểm tra lại thời hạn tại thời điểm xử lý request, do đó học sinh không thể chỉ sửa đồng hồ phía trình duyệt để kéo dài thời gian làm bài. Ngoài ra, giao diện phòng làm bài còn ghi nhận một số hành vi rời khỏi môi trường làm bài như thoát toàn màn hình hoặc chuyển tab nhiều lần; khi số lần vi phạm vượt ngưỡng, hệ thống tự động nộp bài để giảm gian lận trong các bài kiểm tra nghiêm túc.

Khi học sinh nộp bài, hệ thống tự chấm các loại câu khách quan. Với trắc nghiệm, hệ thống so sánh tập option học sinh chọn với tập option đúng. Với câu đúng sai, điểm được tính theo tỷ lệ số mệnh đề trả lời đúng:

$$\text{point} = \text{maxScore} \times \frac{\text{correctCount}}{\text{totalStatements}} \quad (6)$$

Với câu số học, hệ thống so sánh giá trị số học của học sinh với đáp án đúng bằng phép so sánh số thập phân, tránh lỗi cộng điểm do biểu diễn số thực. Tổng điểm tự động được cộng từ điểm của các item khách quan. Nếu bài kiểm tra có câu tự luận, submission chuyển sang trạng thái `submitted` và chờ giáo viên chấm. Nếu không có câu tự luận, submission chuyển sang `completed` và điểm cuối cùng bằng điểm tự động.

0.2.3 Kết quả đạt được và đánh giá

Giải pháp giúp hệ thống bài kiểm tra bao phủ được ba bối cảnh sử dụng mà vẫn dùng chung một mô hình thiết kế. Bài luyện tập công khai, bài kiểm tra trong khóa học và quiz trong bài học khác nhau ở placement, không khác nhau ở lõi lưu đề và lưu bài làm. Nhờ đó, giáo viên có thể tái sử dụng luồng tạo đề, còn học sinh có trải nghiệm nhất quán khi làm bài ở nhiều vị trí khác nhau trong hệ thống.

Về trải nghiệm thực tế, hệ thống cho phép học sinh quay lại lượt làm đang dở nếu submission vẫn còn hiệu lực. Các đáp án đã lưu được khôi phục từ server; các

thay đổi chưa lưu được giao diện cảnh báo trước khi rời trang. Đây là điểm quan trọng đối với học sinh phổ thông vì lỗi mạng, đóng nhầm trình duyệt hoặc chuyển thiết bị là tình huống dễ xảy ra khi làm bài trực tuyến.

Các nhóm test case quan trọng để đánh giá giải pháp gồm: học sinh chưa ghi danh không mở được bài kiểm tra trong khóa học; học sinh không thể làm quá số lượt tối đa; mở lại bài đang làm trả về submission cũ; bài đã nộp không được lưu tiếp; câu trả lời gửi lên phải thuộc đúng item trong đề; option được chọn phải thuộc đúng câu hỏi; hết giờ thì bài được nộp theo trạng thái hiện tại; bài có tự luận chuyển sang trạng thái chờ chấm; bài chỉ có câu khách quan được hoàn tất và có điểm tự động. Các test case này phản ánh đúng các rủi ro nghiệp vụ của hệ thống bài kiểm tra, thay vì chỉ kiểm tra luồng làm bài thành công.

0.3 Xử lý video bài giảng bằng HLS và lưu trữ đối tượng

Video bài giảng là loại dữ liệu có ảnh hưởng trực tiếp tới trải nghiệm học tập và khả năng vận hành thương mại của hệ thống. Nếu học sinh mua khóa học nhưng video tải chậm, khó tua hoặc hay gián đoạn, chất lượng khóa học sẽ bị đánh giá thấp dù nội dung chuyên môn tốt. Phần này trình bày giải pháp xử lý video bằng HLS kết hợp Cloudflare R2, giúp hệ thống giảm tải cho backend, chuẩn hóa định dạng phát và chuẩn bị tốt hơn cho tình huống nhiều học sinh xem cùng lúc.

0.3.1 Bài toán và vấn đề

Trong mô hình dạy học livestream, giáo viên thường có nhiều video ghi lại bài giảng. Một video ôn thi có thể kéo dài hàng chục phút đến vài giờ, dung lượng lớn và được nhiều học sinh xem lại trong cùng một giai đoạn cao điểm trước kỳ thi. Nếu backend lưu trực tiếp các video này trên ổ đĩa máy chủ và truyền nguyên tệp video cho từng học sinh, hệ thống sẽ gặp áp lực lớn về dung lượng, băng thông và số kết nối kéo dài. Khi nhiều học sinh cùng xem, backend dễ bị nghẽn ở phần truyền media, trong khi các API khác như đăng nhập, ghi nhận tiến độ hoặc hỏi AI vẫn cần phản hồi nhanh.

Ngoài vấn đề tải đồng thời, phát video dài trực tiếp từ một tệp nguồn cũng làm trải nghiệm học tập kém ổn định. Học sinh thường tua đến đúng đoạn cần ôn lại, học trên mạng yếu hoặc chuyển giữa nhiều thiết bị. Nếu trình duyệt phải xử lý một tệp video lớn, thao tác tua và tải lại sau gián đoạn sẽ nặng hơn. Vì vậy, hệ thống cần chuyển video sang định dạng phù hợp với phát trực tuyến, chia nhỏ nội dung để trình duyệt tải từng phần, đồng thời đưa tệp media ra khỏi backend ứng dụng.

0.3.2 Giải pháp

Giải pháp được triển khai theo hướng pipeline xử lý bất đồng bộ. Backend không coi video là dữ liệu nghiệp vụ thông thường, mà quản lý video qua metadata, trạng thái xử lý và đường dẫn object. Nội dung video được lưu trong kho đối tượng, còn backend tập trung vào xác thực, kiểm tra quyền học và cung cấp đường truy cập hợp lệ.

a, Tách metadata và object. Khi người quản lý nội dung upload video, hệ thống tạo bản ghi `media` để lưu loại media, trạng thái, object key, MIME type, dung lượng, thời lượng và người tải lên. Nội dung tập được lưu trên Cloudflare R2, một dịch vụ lưu trữ đối tượng tương thích S3. Backend chỉ giữ object key và metadata cần thiết để quản lý media. Cách thiết kế này giúp backend không phụ thuộc vào ổ đĩa cục bộ để lưu video lâu dài.

b, Khóa media trước khi xử lý. Sau khi video được upload thành công, hệ thống bắt đầu quá trình chuyển đổi HLS. Trước khi chạy FFmpeg, dịch vụ transcode gọi thao tác khóa media. Nếu media đã bị xóa, không phải video hoặc đã có tiến trình xử lý khác lấy được khóa, dịch vụ dừng lại. Cơ chế này tránh tình huống cùng một video bị xử lý song song, gây ghi đè playlist hoặc tạo nhiều kết quả không nhất quán.

c, Chuyển đổi sang HLS. Dịch vụ xử lý tạo thư mục tạm riêng cho media, tải video nguồn từ R2 về máy chủ, sau đó dùng FFprobe để đọc thời lượng. Tiếp theo, FFmpeg được gọi để chuyển video sang HLS với codec video H.264, audio AAC, thời lượng segment 6 giây và playlist dạng VOD. Kết quả gồm một tệp `index.m3u8` và nhiều segment `segment_xxx.ts`. Playlist mô tả thứ tự các segment, còn trình duyệt chỉ tải những segment cần phát tại thời điểm hiện tại. Khi học sinh tua video, trình phát có thể chuyển đến nhóm segment tương ứng thay vì tải lại toàn bộ tệp nguồn.

d, Upload kết quả và cập nhật trạng thái. Sau khi FFmpeg xử lý xong, hệ thống upload toàn bộ playlist và segment lên R2 trong thư mục HLS riêng của media. Nếu thành công, bản ghi media được cập nhật sang trạng thái sẵn sàng, lưu object key của playlist và thời lượng video. Nếu bất kỳ bước nào lỗi, media được đánh dấu thất bại. Dù thành công hay thất bại, hệ thống đều xóa thư mục tạm để tránh làm đầy ổ đĩa máy chủ.

e, Phục vụ video qua route có kiểm soát. Khi học sinh học bài, frontend không truy cập tùy ý vào toàn bộ bucket. Backend cung cấp route lấy playlist và segment HLS. Route này chỉ chấp nhận tên tệp đúng mẫu playlist hoặc segment. Với playlist và segment, backend trả về đúng kiểu nội dung tương ứng để trình phát video xử

lý. Việc giới hạn tên tệp giúp tránh truy cập ngoài phạm vi video bài học, đồng thời tạo điểm kiểm soát để kết hợp kiểm tra quyền học.

Về mặt tải hệ thống, HLS giúp mỗi lượt xem được chia thành nhiều request nhỏ theo từng segment. Thiết kế này phù hợp với cache và CDN hơn so với truyền một tệp video lớn qua backend. Trong phạm vi đồ án, Cloudflare R2 được dùng để lưu trữ object lâu dài; khi triển khai thực tế, các segment HLS có thể được phục vụ qua lớp cache ở gần người học hơn, giúp giảm độ trễ tải video và giảm áp lực băng thông cho máy chủ ứng dụng.

Ở mức API, route phát HLS có dạng:

```
GET /learning/lessons/:lessonId/hls/:fileName
```

Tham số `lessonId` xác định bài học đang xem, còn `fileName` chỉ được nhận hai dạng: `index.m3u8` hoặc `segment_xxx.ts`. Khi nhận request, service `getLearningLessonHlsFile` kiểm tra học sinh có quyền học bài đó, bài học có video hệ thống, media đã ở trạng thái `ready`, sau đó mới đọc object từ R2. Nếu `fileName` là `playlist`, backend sửa lại các đường dẫn segment trong playlist để các request tiếp theo vẫn đi qua route có kiểm soát. Nếu `fileName` là `segment`, backend stream nội dung segment về trình duyệt với `Content-Type` phù hợp.

0.3.3 Kết quả đạt được và đánh giá

Giải pháp HLS và lưu trữ đối tượng giúp hệ thống xử lý video theo hướng phù hợp hơn với vận hành thực tế. Backend không còn là nơi lưu giữ lâu dài các tệp video lớn, mà chỉ quản lý metadata, trạng thái và quyền truy cập. Video sau xử lý được chuẩn hóa thành playlist và segment, thuận lợi hơn cho phát trên trình duyệt, tua video và phục vụ nhiều lượt xem cùng lúc.

Quy trình này cũng có khả năng xử lý lỗi rõ ràng. Nếu nguồn video không tồn tại, FFmpeg lỗi hoặc không tạo được segment, media không bị đánh dấu sẵn sàng sai mà chuyển sang trạng thái thất bại. Người quản lý nội dung có thể nhận biết video chưa dùng được. Ngoài ra, cơ chế khóa media và cleanup thư mục tạm giúp tránh hai lỗi thường gặp trong xử lý media: chạy trùng job và để lại tệp tạm sau khi xử lý.

Các tiêu chí đánh giá giải pháp gồm: video upload chuyển sang trạng thái xử lý; sau khi transcode thành công có playlist và các segment trên R2; thời lượng video được cập nhật; trình phát tải được playlist và segment đúng kiểu nội dung; học sinh có thể tua đến các đoạn khác nhau của video; route học tập chỉ nhận playlist hoặc segment hợp lệ; media lỗi được đánh dấu `failed`; thư mục tạm được xóa sau xử

lý. Các tiêu chí này phản ánh đúng rủi ro của luồng video bài giảng trong một LMS có doanh thu từ khóa học, thay vì chỉ kiểm tra việc upload một tệp thành công.

0.4 Tự động hóa ghi danh sau thanh toán qua webhook

Một mục tiêu thực tế của hệ thống là giảm thao tác vận hành thủ công khi bán khóa học. Trong mô hình cũ, giáo viên hoặc trợ lý phải kiểm tra chuyển khoản, đối chiếu nội dung chuyển khoản rồi thêm học sinh vào nhóm học. Phần này trình bày giải pháp tự động hóa ghi danh sau thanh toán bằng đơn hàng, VietQR, webhook, cơ chế chống request trùng và đối soát giao dịch.

0.4.1 Bài toán và vấn đề

Thanh toán chuyển khoản trong bối cảnh lớp học trực tuyến không chỉ có trường hợp thành công đơn giản. Học sinh có thể bấm mua nhiều lần do mạng chậm, làm mới trang trong lúc tạo thanh toán, chuyển khoản thiếu tiền, chuyển sau khi đơn hàng hết hạn, nhập thiếu mã hóa đơn hoặc chuyển nhầm nội dung. Ở phía provider, webhook có thể được gửi lặp lại nếu lần gửi trước chưa nhận được phản hồi thành công. Nếu hệ thống tự động cấp quyền học cho mọi webhook có trạng thái thành công, khóa học có thể bị cấp sai. Ngược lại, nếu hệ thống chỉ dựa vào kiểm tra thủ công, mục tiêu giảm vận hành không đạt được.

Bài toán cần giải quyết là thiết kế một luồng tự động hóa nhưng vẫn có kiểm soát. Hệ thống phải hạn chế request trùng từ phía người dùng, tự động ghi danh khi giao dịch khớp chính xác, đồng thời giữ lại dữ liệu để quản trị viên xử lý các giao dịch bất thường. Ngoài ra, quá trình cập nhật thanh toán, hoàn tất đơn hàng và tạo enrollment phải nhất quán; không được xảy ra tình trạng đơn hàng đã hoàn thành nhưng chưa có quyền học, hoặc có quyền học khi payment chưa hợp lệ.

0.4.2 Giải pháp

Giải pháp được triển khai theo mô hình đơn hàng trước, thanh toán sau. Mọi giao dịch thanh toán đều được đối chiếu với một đơn hàng đã tồn tại trong hệ thống.

a, Chống request trùng bằng idempotency key. Các thao tác nhạy cảm như tạo đơn hàng, tạo thanh toán và hủy đơn đều yêu cầu Idempotency-Key. Khi request đầu tiên đến, hệ thống lưu key cùng hash của method, đường dẫn và body request. Nếu cùng key và cùng nội dung request được gửi lại sau khi xử lý xong, hệ thống trả lại response đã lưu thay vì thực hiện nghiệp vụ lần nữa. Nếu cùng key nhưng nội dung request khác, hệ thống từ chối vì đây không còn là retry hợp lệ. Nếu request cũ vẫn đang xử lý, hệ thống trả lỗi đang xử lý để tránh hai tiến trình cùng tạo dữ liệu.

Cơ chế này quan trọng trong luồng mua khóa học vì người dùng có thể bấm nút

thanh toán nhiều lần khi mạng chậm. Nếu chỉ dựa vào ràng buộc duy nhất ở CSDL, hệ thống vẫn phải để nhiều request đi sâu vào logic nghiệp vụ rồi mới bị chặn ở tầng cuối. Idempotency giúp chặn sớm hơn, làm rõ ý nghĩa của request lặp và trả về kết quả ổn định cho frontend.

b, Tạo đơn hàng và mã hóa đơn. Khi học sinh mua khóa học, hệ thống loại bỏ các khóa học trùng trong request, kiểm tra khóa học đã xuất bản, kiểm tra học sinh chưa ghi danh các khóa học đó và tính tổng tiền. Trước khi tạo đơn mới, hệ thống làm hết hạn các đơn chờ quá thời gian và kiểm tra xem học sinh có đơn pending còn hiệu lực hay không. Nếu có, hệ thống trả về đơn đang chờ thay vì tạo thêm đơn mới. Cách làm này tránh việc một học sinh tạo nhiều đơn chờ song song cho cùng một nhu cầu mua.

Mỗi đơn hàng có mã hóa đơn duy nhất `order_invoice_number`. Khi tạo thanh toán qua SePay, backend sinh mã VietQR gồm số tài khoản, ngân hàng, số tiền và nội dung chuyển khoản là mã hóa đơn. Mã hóa đơn là dữ liệu quan trọng để webhook ngân hàng có thể ghép giao dịch với đơn hàng.

c, Xác thực và chuẩn hóa webhook. Khi SePay gửi webhook, provider adapter kiểm tra request theo cấu hình của hệ thống. Hệ thống hỗ trợ xác thực bằng API key hoặc chữ ký HMAC. Với HMAC, adapter kiểm tra timestamp để tránh replay trong khoảng thời gian quá xa, sau đó so sánh chữ ký bằng `timingSafeEqual`. Sau khi request hợp lệ, payload được chuẩn hóa thành sự kiện thanh toán nội bộ gồm provider, event id, mã giao dịch, mã hóa đơn, số tiền, đơn vị tiền tệ, hướng giao dịch, trạng thái, thời điểm thanh toán và raw payload.

Payload webhook từ SePay có nhiều trường gắn với giao dịch ngân hàng như mã sự kiện, ngân hàng, thời gian giao dịch, số tài khoản, mã đơn hàng, hướng chuyển tiền, số tiền và mã tham chiếu. Trong hệ thống, provider adapter không để các service nghiệp vụ phụ thuộc trực tiếp vào payload thô này, mà chuẩn hóa thành cấu trúc sự kiện nội bộ:

```
{
  "provider": "sepay",
  "eventId": "...",
  "transactionRef": "...",
  "orderInvoiceNumber": "...",
  "amount": 200000,
  "currency": "VND",
  "status": "success",
  "direction": "in",
```

```
"paidAt": "..."  
}
```

Trong đó, `orderInvoiceNumber` được lấy từ trường `code` của SePay và dùng để ghép giao dịch với đơn hàng. Việc chuẩn hóa này giúp service xử lý web-hook chỉ làm việc với một dạng dữ liệu ổn định, kể cả khi provider thay đổi tên trường hoặc hệ thống bổ sung provider thanh toán khác trong tương lai.

d, Lưu webhook event và payment transaction. Trước khi đối chiếu, hệ thống lưu webhook event và tạo bản ghi payment transaction. Hai nhóm dữ liệu này có vai trò khác nhau. Webhook event lưu sự kiện do provider gửi, phục vụ chống xử lý lặp và theo dõi trạng thái xử lý webhook. Payment transaction lưu giao dịch tiền thực tế, phục vụ đối soát, tìm kiếm và xử lý thủ công. Việc tách hai loại dữ liệu giúp hệ thống không mất dấu các giao dịch bất thường.

e, Đối soát giao dịch. Sau khi tạo bản ghi giao dịch, hệ thống phân loại theo điều kiện nghiệp vụ. Nếu webhook không phải giao dịch tiền vào hoặc không có trạng thái thành công, giao dịch được đánh dấu `ignored`. Nếu thiếu mã hóa đơn hoặc không tìm thấy đơn hàng, giao dịch được đánh dấu `unmatched`. Nếu đơn hàng tồn tại nhưng đơn vị tiền tệ không khớp, giao dịch được đưa vào diện cần kiểm tra thủ công.

Với đơn hàng tìm thấy, hệ thống tiếp tục đánh giá trạng thái payment. Nếu payment không còn `pending`, giao dịch được đưa vào `manual_review` với lý do `payment_not_pending`. Nếu số tiền nhận được nhỏ hơn tổng tiền đơn hàng, giao dịch được đưa vào `manual_review` với lý do `under_paid`. Nếu đơn hàng đã hết hạn hoặc không còn ở trạng thái `pending`, giao dịch được ghi nhận là `late_success`. Chỉ khi payment đang chờ, số tiền đủ, đơn chưa hết hạn và đơn còn `pending`, hệ thống mới coi giao dịch là hợp lệ.

Bảng 1: Các nhóm kết quả đối chiếu giao dịch thanh toán

Trạng thái	Ý nghĩa
matched	Giao dịch khớp với đơn hàng hợp lệ, hệ thống hoàn tất đơn hàng và cấp quyền học.
unmatched	Giao dịch thiếu mã hóa đơn hoặc không tìm thấy đơn hàng tương ứng.
manual_review	Giao dịch cần quản trị viên kiểm tra lại, ví dụ thiếu tiền, đơn hàng hết hạn hoặc payment không còn ở trạng thái chờ.
ignored	Giao dịch không phải tiền vào hoặc không phải trạng thái thành công nên không tham gia cấp quyền học.

f, Hoàn tất đơn hàng trong transaction. Khi giao dịch hợp lệ, hệ thống cập nhật payment sang `success`, cập nhật payment transaction sang `matched`, hoàn tất đơn hàng và tạo các bản ghi `enrollments` cho các khóa học trong đơn. Toàn bộ thao tác được thực hiện trong transaction CSDL. Nếu một bước lỗi, transaction rollback, tránh tình trạng dữ liệu thanh toán và quyền học lệch nhau.

Hệ thống cũng xử lý webhook gửi lặp bằng ràng buộc duy nhất theo provider và event id. Nếu provider gửi lại một webhook đã xử lý, hệ thống nhận ra sự kiện cũ và không tạo thêm enrollment. Đây là điểm quan trọng vì webhook từ bên thứ ba thường có cơ chế retry khi không nhận được phản hồi thành công.

0.4.3 Kết quả đạt được và đánh giá

Giải pháp giúp tự động hóa phần lớn luồng ghi danh sau thanh toán. Với giao dịch khớp chính xác, học sinh được cấp quyền học mà không cần giáo viên hoặc trợ lý kiểm tra biên lai thủ công. Với giao dịch bất thường, hệ thống không tự động cấp quyền nhưng vẫn lưu đủ dữ liệu để quản trị viên rà soát.

Giải pháp cũng cải thiện tính nhất quán dữ liệu. Cấp quyền học không phải một thao tác rời rạc, mà gắn với trạng thái đơn hàng và payment trong cùng transaction. Điều này phù hợp với yêu cầu thực tế của hệ thống thương mại hóa khóa học: học sinh chỉ được ghi danh khi đơn hàng đã hoàn tất hợp lệ.

Về phía người dùng, idempotency key giúp thao tác mua khóa học ổn định hơn trong các tình huống bấm lặp hoặc retry do mạng chậm. Về phía quản trị, webhook event và payment transaction tạo thành dữ liệu đối soát: quản trị viên có thể biết giao dịch nào khớp, giao dịch nào thiếu mã hóa đơn, giao dịch nào đến muộn hoặc thiếu tiền. Đây là điểm quan trọng vì hệ thống bán khóa học không chỉ cần cấp quyền nhanh, mà còn cần tránh cấp quyền sai.

Các test case quan trọng gồm: gửi lặp request tạo đơn với cùng idempotency key trả lại cùng kết quả; cùng key nhưng body khác bị từ chối; không tạo đơn mới nếu còn đơn pending hiệu lực; tạo QR với mã hóa đơn; webhook hợp lệ tạo enrollment; webhook gửi lặp không tạo enrollment trùng; giao dịch thiếu mã hóa đơn được đánh dấu unmatched; giao dịch thiếu tiền được đưa vào manual review; giao dịch đến sau khi đơn hết hạn không tự động cấp quyền; giao dịch tiền ra bị ignored. Các trường hợp này cho thấy hệ thống không chỉ xử lý luồng thành công, mà còn bao phủ các lỗi thường gặp của thanh toán chuyển khoản tự động.

Chương này đã trình bày bốn đóng góp kỹ thuật nổi bật của đề án: trợ lý AI hỏi đáp theo học liệu nội bộ có nguồn dẫn, kiểm soát quá trình làm bài kiểm tra, xử lý video bài giảng bằng HLS kết hợp lưu trữ đối tượng và tự động hóa ghi danh sau

thanh toán qua webhook. Các giải pháp này trực tiếp giải quyết các vấn đề đã nêu ở Chương 1 và Chương 2, đồng thời thể hiện cách áp dụng các công nghệ ở Chương 3 vào các bài toán triển khai cụ thể. Chương tiếp theo sẽ tổng kết kết quả đạt được, các hạn chế còn tồn tại và hướng phát triển của hệ thống.